



Programação para Web 1

**Nota de aula: Site pessoal
com HTML, CSS e
layout responsivo**

Prof. Bruno Honorato

Outubro, 2026

Sumário

1. Projeto incremental e roteiro de estudo	4
2. Etapa 1: HTML puro	5
2.1. Objetivo e requisitos da primeira etapa	5
2.2. Estrutura básica do documento	5
2.3. Cabeçalho, imagem e navegação	6
2.4. Conteúdo principal e hierarquia semântica	6
2.5. Links e contato	7
2.6. Formulário de demonstração	7
2.7. Identificadores, seletores e lógica do layout	8
2.8. Execução, validação e entrega	8
3. Etapa 2: CSS externo	10
3.1. Objetivo e requisitos	10
3.2. Arquivos e execução	10
3.3. Evolução do HTML	10
3.4. Lógica da apresentação	11
3.5. Por que usar cada seletor	11
3.6. Cascata, especificidade e herança	14
3.7. Layout e limite desta etapa	14
3.8. Conferência e entrega	15
4. Etapa 3: Mobile First e CSS Grid	16
4.1. Objetivo e requisitos	16
4.2. Arquivos e execução	16
4.3. HTML e viewport	16
4.4. Mobile First e breakpoints	17
4.5. Por que a grade funciona	18

4.6. Seletores e finalidade de cada regra	19
4.7. Unidades, conteúdo e acessibilidade	22
4.8. Validação em três larguras	22
5. Código para consulta	24
5.1. HTML da etapa 1	25
5.2. CSS da etapa 2	27
5.3. CSS da etapa 3	31

1. Projeto incremental e roteiro de estudo

O objetivo é construir um site pessoal em três versões: estrutura semântica em HTML puro, apresentação com CSS externo e adaptação Mobile First com CSS Grid. Cada versão fica em uma pasta independente, permitindo observar o que muda sem perder a etapa anterior.

Na primeira etapa, explique a escolha de cada tag e atributo. Na segunda, associe cada regra CSS ao elemento que ela seleciona e à mudança visual que produz. Na terceira, justifique os contêineres da grade, as colunas, os intervalos e os breakpoints. O conteúdo e a ordem do documento permanecem coerentes ao longo das versões.

Etapa	Pasta	Foco
1	01-html-puro	Documento, semântica, imagem, navegação e formulário
2	02-css	Cores, tipografia, modelo de caixa, seletores e estados
3	03-mobile-first	Viewport, unidades flexíveis, media queries e CSS Grid

Abra o `index.html` de cada pasta para executar os exemplos. Não há frameworks ou JavaScript. O formulário é demonstrativo e não possui um serviço de envio. A publicação e a conferência visual do site são atividades previstas no roteiro de validação, não resultados já concluídos.

2. Etapa 1: HTML puro

2.1. Objetivo e requisitos da primeira etapa

O objetivo é construir a primeira versão do site pessoal de Bruno de Castro Honorato Silva usando somente HTML. Esta etapa não permite CSS, frameworks CSS ou JavaScript. A aparência simples é intencional: avaliamos o significado dos elementos, a organização do conteúdo e a navegação.

O exemplo atende aos itens mínimos: nome e apresentação, fotografia, Sobre mim, formação acadêmica, experiências e projetos, habilidades e tecnologias, perfis profissionais, contato e navegação entre as partes. Há também um formulário demonstrativo. O enunciado aceita GitHub, LinkedIn ou outros perfis relevantes; o exemplo inclui LinkedIn, Lattes e ORCID.

Abra `index.html` diretamente no navegador. A foto está em `imagens/bruno-honorato.jpg`; portanto, mantenha essa pasta junto do HTML. Não há dependências para executar o site.

2.2. Estrutura básica do documento

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Site pessoal de Bruno de Castro Honorato Silva,
  professor da UTFPR: formação, pesquisa, tecnologias e contato.">
  <title>Bruno Honorato | Site pessoal</title>
</head>
<body>
  <!-- O conteúdo visível fica aqui. -->
</body>
</html>
```

`<!DOCTYPE html>` é uma declaração, não uma tag de conteúdo. Ela informa que o documento usa HTML moderno e aciona o modo de padrões do navegador.

`html` é o elemento raiz. `lang="pt-BR"` identifica o idioma, ajudando leitores de tela a pronunciar o texto corretamente. `head` reúne metadados; `body` contém o conteúdo apresentado ao visitante. Separá-los evita colocar informações de configuração no fluxo da página.

As três ocorrências de `meta` possuem finalidades distintas. `charset="UTF-8"` preserva os caracteres acentuados. `name="viewport"` e seu `content` ajustam a área de visualização à largura do dispositivo e definem a escala inicial; essa configuração prepara as etapas seguintes, mas não cria responsividade por si só. `name="description"` e seu `content` descrevem a página para ferramentas que leem metadados, podendo ser usados em resultados de busca.

`title` define o título da aba e dos favoritos. Ele não substitui o título visível `h1`. Não usamos `style`, `link` para folhas de estilo, atributos `style` nem `script`, porque CSS e JavaScript estão fora do escopo.

2.3. Cabeçalho, imagem e navegação

header reúne a apresentação e a navegação introdutória da página. Seu `id="inicio"` permite voltar ao topo pelo link do rodapé. O `h1` contém o nome completo: é o título principal desta página. `p` marca a breve apresentação como parágrafo, em vez de simular títulos ou espaçamento com quebras de linha.

`figure` agrupa a fotografia e sua legenda como uma unidade de conteúdo. `img` incorpora a imagem; `src` aponta para um arquivo local relativo ao HTML. `alt` oferece uma descrição textual do retrato para quem não consegue vê-lo. `width="240"` e `height="240"` fornecem dimensões de apresentação em pixels e preservam a proporção quadrada do arquivo original. São atributos HTML da imagem, não CSS. Eles não constituem uma solução geral de imagem responsiva. `figcaption` identifica o professor na legenda visível, complementando o texto alternativo.

```
<nav aria-label="Navegação principal">
  <ul>
    <li><a href="#sobre">Sobre mim</a></li>
    <li><a href="#formacao">Formação acadêmica</a></li>
  </ul>
</nav>
```

`nav` identifica um conjunto importante de links de navegação. `aria-label` dá um nome acessível a essa região. `ul` representa um conjunto de destinos sem ordem obrigatória; `li` identifica cada item. `a` cria o hyperlink; `href="#sobre"` aponta para o elemento com `id="sobre"`. O mesmo raciocínio se aplica a formação, experiências, habilidades, perfis e contato. O deslocamento é nativo do navegador: não exige JavaScript.

2.4. Conteúdo principal e hierarquia semântica

`main` contém o assunto central da página. Há uma única ocorrência, separada do cabeçalho e do rodapé. `section` divide esse assunto em seis blocos temáticos: Sobre mim, Formação acadêmica, Experiências e projetos, Habilidades e tecnologias, Perfis profissionais e Contato. Cada seção tem um `h2` e um identificador exclusivo.

Os títulos seguem uma hierarquia: `h1` identifica o site pessoal, `h2` apresenta as seções e `h3` nomeia cada experiência dentro da seção de projetos. Escolhemos o nível pelo papel no documento, não pelo tamanho visual padrão. Isso facilita a leitura e a navegação por títulos em tecnologias assistivas.

`article` aparece em cada experiência porque cada bloco possui um título e uma descrição que fazem sentido como uma unidade independente. `section` agrupa o tema; `article` individualiza um relato dentro dele. Não usamos `article` para todo parágrafo nem criamos `div` sem necessidade.

`p` separa as ideias da apresentação, da biografia e dos projetos. `ul` e `li` também organizam formações, habilidades e perfis: são coleções, não sequências de instruções. `strong` destaca semanticamente os nomes dos graus acadêmicos. Sua escolha indica importância, embora o navegador normalmente também mostre o texto em negrito.

O conteúdo apresenta as instituições, os períodos de formação e as áreas de atuação. A seção de tecnologias não atribui níveis de proficiência ou certificações adicionais.

2.5. Links e contato

Nos perfis, a usa endereços HTTPS completos em href, porque o destino é externo ao site. O texto descreve o destino, como “LinkedIn de Bruno Honorato”, em vez de “clique aqui”. Os links abrem na mesma aba pelo comportamento padrão; não há atributo target.

address marca a informação de contato profissional do autor da página. Ele não é um recurso para colocar qualquer endereço postal em itálico. br separa a instituição da linha de e-mail, uma quebra pertinente ao bloco de contato. Não o usamos para fabricar margens ou alinhar colunas.

O link com href="mailto:brunosilva@utfpr.edu.br" solicita ao navegador a abertura de um aplicativo de e-mail configurado. O envio depende desse aplicativo; o site não envia mensagens sozinho.

footer encerra a página com autoria, contexto didático e o link href="#inicio". Sua posição após main expressa o encerramento do documento; o elemento não fixa o rodapé na parte inferior da janela.

2.6. Formulário de demonstração

```
<form>
  <fieldset>
    <legend>Mensagem de contato (demonstração)</legend>
    <p><label for="nome">Nome:</label><br><input type="text" id="nome" name="nome"
autocomplete="name" required></p>
    <p><label for="email">E-mail:</label><br><input type="email" id="email"
name="email" autocomplete="email" required></p>
    <p><label for="mensagem">Mensagem:</label><br><textarea id="mensagem"
name="mensagem" rows="5" cols="30" required></textarea></p>
    <button type="button" disabled>Envio indisponível nesta etapa</button>
  </fieldset>
</form>
```

form identifica a região de coleta de dados. Não configuramos action nem serviço de processamento nesta etapa: o formulário é demonstrativo, como permite o enunciado. A omissão de action não deve ser interpretada como garantia de ausência de submissão; se houver submissão nativa, o destino padrão é o documento atual. O exemplo evita oferecer um botão de envio e orienta o visitante a usar o e-mail.

fieldset agrupa os controles relacionados e legend fornece o título acessível desse grupo. label dá um nome a cada campo. Seu for deve coincidir exatamente com o id do controle: nome, email e mensagem. Clicar no rótulo transfere o foco ao campo correspondente.

input é usado para valores de uma linha. type="text" aceita o nome; type="email" expressa o formato esperado e permite verificações nativas de formato quando a validação é acionada. Essas verificações não comprovam que o endereço existe. textarea aceita a mensagem em várias linhas. rows="5" e cols="30" sugerem dimensões iniciais em linhas e colunas de caracteres, não limites de tamanho da mensagem.

`id` identifica cada controle dentro do documento; `name` define a chave que seria usada na submissão dos dados. São finalidades diferentes mesmo quando os valores são iguais. `autocomplete="name"` e `autocomplete="email"` informam o tipo de dado para o preenchimento automático do navegador. `required` marca campos obrigatórios para uma futura submissão válida; o botão deste exemplo não aciona essa validação.

`button` exhibe explicitamente a indisponibilidade do envio. `type="button"` evita a função padrão de submissão e `disabled` impede sua ativação. Não há promessa de envio, armazenamento ou confirmação. Os `p` agrupam cada rótulo com seu controle, e `br` separa o rótulo da área de entrada na apresentação nativa.

2.7. Identificadores, seletores e lógica do layout

Nesta etapa não há seletores CSS nem layout em grade. Isso é uma decisão exigida pelo enunciado da primeira entrega, não uma lacuna do exemplo. O navegador apresenta os elementos com seus estilos padrão e o documento segue a ordem de leitura do HTML: apresentação, navegação, seções e rodapé. As listas e os blocos ficam no fluxo normal, sem posicionamento manual.

`id="sobre"` é um atributo HTML. `href="#sobre"` é uma referência a um fragmento do documento. Já `#sobre`, dentro de uma futura regra CSS, seria um seletor de ID. A grafia parecida não transforma a âncora em CSS. Os IDs das seções são destinos de navegação; os IDs dos controles são alvos dos rótulos. Todos devem ser únicos.

Também não há atributos `class` no exemplo. Não acrescentamos classes apenas para antecipar um CSS que não existe. Na etapa 2, cada seletor introduzido deverá ser explicado pelo conjunto de elementos que seleciona e pelo efeito da regra correspondente.

O layout em grade pertence à etapa 3. Uma futura aplicação de Grid deverá justificar qual elemento vira contêiner, quais filhos são itens, como as colunas são definidas, por que determinado espaçamento é usado e quando a quantidade de colunas muda. A ordem visual deverá preservar uma leitura coerente com a ordem do HTML. Essas regras não estão implementadas na primeira etapa.

Não usamos tabelas, espaços repetidos nem atributos antigos de apresentação para simular uma grade. Tabelas representam dados tabulares; a organização de colunas de uma interface será tratada com CSS nas próximas etapas.

2.8. Execução, validação e entrega

1. Abra `index.html` no navegador e confirme nome, apresentação, foto e todas as seções.
2. Clique nos seis links do menu e em “Voltar ao início”. Cada fragmento deve corresponder a um ID existente e exclusivo.
3. Acesse os links de Lattes, LinkedIn e ORCID. A disponibilidade e as exigências de login pertencem aos serviços externos.

4. Clique nos rótulos do formulário e navegue pelos controles com a tecla Tab. Verifique que o botão informa a indisponibilidade e está desabilitado.
5. Confira que a página não inclui CSS, JavaScript ou frameworks e que o arquivo de imagem foi incluído na entrega.
6. Valide o HTML em um validador de conformidade, como o Nu HTML Checker, antes da publicação. A inspeção local da estrutura não substitui essa validação completa.
7. Faça commit do código, da imagem e do README no repositório da atividade. Configure o GitHub Pages e registre na documentação da etapa a URL real depois de publicá-lo.

Aluno/autor do exemplo: Bruno de Castro Honorato Silva. Objetivo: demonstrar a primeira entrega do site pessoal com estrutura semântica e conteúdo em HTML puro. Publicação: pendente; a URL de GitHub Pages ainda não foi registrada. A publicação não foi realizada nesta tarefa.

Como este exemplo está em uma subpasta, uma publicação do repositório inteiro normalmente usará o caminho `/07-trabalho-1/01-html-puro/` após a URL base do GitHub Pages. Confirme a configuração real do repositório antes de registrar o endereço.

3. Etapa 2: CSS externo

3.1. Objetivo e requisitos

Esta etapa transforma visualmente a página de 01-html-puro. O conteúdo e a estrutura continuam em HTML; a apresentação fica em uma folha CSS externa. O exemplo contempla cores, tipografia, espaçamentos, bordas, tamanhos, links, formulário, organização visual das seções e estado hover.

Não há JavaScript ou frameworks. Grid, Flexbox e media queries serão introduzidos na etapa 3. Esta etapa concentra-se na apresentação visual.

3.2. Arquivos e execução

```
02-css/  
|-- index.html  
|-- estilos.css  
|-- imagens/  
|   +-- bruno-honorato.jpg  
+-- README.md
```

Abra index.html no navegador. Não é necessário instalar dependências. A fotografia está copiada nesta etapa para que ela possa ser executada sem depender dos arquivos da etapa 1. Mantenha os caminhos relativos ao mover ou publicar o exemplo.

Aluno/autor do exemplo: Bruno de Castro Honorato Silva. Objetivo: separar estrutura e conteúdo da apresentação por meio de CSS externo. Link do site publicado: pendente; a publicação no GitHub Pages não faz parte desta alteração.

3.3. Evolução do HTML

A única nova tag é link, dentro de head:

```
<link rel="stylesheet" href="estilos.css">
```

rel="stylesheet" identifica a relação do recurso com a página: trata-se de uma folha de estilos. href="estilos.css" informa seu caminho relativo ao HTML. link é um elemento vazio, sem tag de fechamento. O título da aba também passa a identificar a versão com CSS.

As demais tags e os atributos são preservados e estão explicados no README da etapa 1. header, nav, main, section, article e footer continuam expressando seus papéis semânticos; não foram substituídos por elementos escolhidos apenas pela aparência. Os IDs continuam servindo às âncoras e aos rótulos do formulário. Não foi necessário adicionar classes para selecionar os elementos deste documento pequeno.

Para comparar as etapas, abra os dois arquivos HTML em abas distintas. Desabilitar a folha de estilos no navegador também revela a estrutura original. O formulário permanece demonstrativo, com envio indisponível: CSS não cria processamento ou envio de dados.

3.4. Lógica da apresentação

A página segue uma única coluna na ordem do HTML. O cabeçalho apresenta nome, fotografia e menu; as seções ficam empilhadas; o rodapé encerra o conteúdo. O fluxo normal do navegador faz essa organização, sem posicionamento absoluto, Flexbox ou Grid.

`max-width: 58rem` limita a largura dos blocos centrais para facilitar a leitura em telas amplas. `margin: 0 auto` centraliza esses blocos horizontalmente quando há espaço disponível. Não usamos altura fixa para o conteúdo: cada bloco cresce de acordo com seu texto.

O cabeçalho azul escuro e o detalhe dourado identificam visualmente a apresentação. Fundo cinza claro e seções brancas separam os assuntos; bordas e espaçamentos reforçam essa divisão. Os projetos têm um fundo próprio dentro da seção de experiências. Essas escolhas visuais complementam a hierarquia semântica existente.

O menu usa `inline-block`: cada item continua sendo uma caixa com padding e borda, mas participa de linhas de texto que podem quebrar quando falta espaço. Isso não é uma grade. As seções permanecem em uma coluna em todas as larguras, e nenhum breakpoint é definido nesta etapa.

`rem` toma como referência o tamanho da fonte do elemento raiz; `em` usa a fonte do elemento em que a propriedade é aplicada. Por exemplo, o afastamento do sublinhado em `0.2em` acompanha o tamanho do link. Percentuais como `width: 100%` se referem à largura disponível no bloco que contém o controle. Bordas de `1px` mantêm um contorno fino.

3.5. Por que usar cada seletor

Esta tabela cobre todos os seletores existentes em `estilos.css`, na ordem do arquivo. Uma vírgula agrupa seletores para compartilhar declarações; um espaço seleciona descendentes; `>` seleciona filhos diretos; `+` seleciona o irmão seguinte imediato. As pseudoclasses representam estados ou posições, sem alterar o HTML.

Seletor	Alvo e motivo das declarações
*	Todos os elementos. <code>box-sizing: border-box</code> inclui padding e borda na largura declarada, evitando que campos com <code>width: 100%</code> ultrapassem o contêiner por causa desses acréscimos.
body	Define a base da página: remove a margem padrão, aplica fundo, cor, família tipográfica, tamanho e altura de linha. <code>overflow-wrap: anywhere</code> permite quebrar palavras ou endereços muito longos quando necessário.
header, main, footer	Compartilha largura máxima, centralização e padding entre os três blocos principais para manter o mesmo eixo visual.
header	Aplica o fundo azul, texto branco, alinhamento central e borda inferior dourada à apresentação.

Seletor	Alvo e motivo das declarações
h1, h2, h3	Reduz a altura de linha dos títulos para manter títulos de várias linhas coesos, independentemente da altura de linha do corpo.
h1	Dá destaque ao nome por tamanho e margem inferior, removendo a margem superior padrão.
header > p	Seleciona apenas o parágrafo de apresentação que é filho direto do cabeçalho. Limita sua largura, centraliza-o e usa uma cor clara. Não afeta parágrafos das seções.
figure	Remove a margem lateral padrão da figura e mantém uma distância inferior até o menu.
figure img	Seleciona a fotografia dentro da figura. <code>display: block</code> permite centralizá-la com margens automáticas. A largura de <code>12rem</code> , <code>max-width: 100%</code> e <code>height: auto</code> controlam tamanho e proporção. Borda dourada e <code>border-radius: 50%</code> tornam o retrato quadrado circular.
figcaption	Separa a legenda da foto, reduz seu tamanho e mantém contraste no fundo escuro.
nav ul	Remove marcadores e recuos apenas da lista do menu. As listas do conteúdo continuam mostrando marcadores.
nav li	Usa <code>inline-block</code> e pequenas margens para organizar itens do menu em linhas com intervalos entre as caixas.
nav a	Amplia a área clicável com padding, cria borda e cantos arredondados e aplica texto branco. Remove o sublinhado porque o contorno já identifica os links como controles de navegação.
nav a:hover	Ao passar o ponteiro sobre um link do menu, inverte fundo e cor do texto para indicar a interação.
main > section	Seleciona as seções diretamente contidas em <code>main</code> . Aplica fundo branco, borda, cantos arredondados, padding interno e distância inferior entre os blocos.
h2	Marca o início das seções com fonte maior, cor azul, margem inferior e uma borda lateral dourada acompanhada de padding.
h3	Identifica os títulos de projetos com tamanho menor que h2, preservando a hierarquia visual.
p	Normaliza o espaçamento dos parágrafos: sem margem superior e com margem inferior.

Seletor	Alvo e motivo das declarações
<code>main ul</code>	Define recuo para os marcadores das listas do conteúdo e remove margens padrão que variam entre elementos.
<code>main li + li</code>	Acrescenta espaço antes de cada item que venha imediatamente após outro <code>li</code> . O primeiro item não recebe esse espaço.
<code>article</code>	Delimita cada experiência com padding, fundo claro e borda lateral. A unidade semântica já existe no HTML.
<code>article + article</code>	Separa experiências consecutivas sem acrescentar espaço antes do primeiro projeto.
<code>article > p</code>	Remove a margem inferior do parágrafo direto de cada projeto; o padding do artigo já fornece espaço até sua borda.
<code>a</code>	Define a cor dos links e afasta o sublinhado do texto para melhorar a leitura. O menu sobrescreve a cor com seu seletor mais específico.
<code>a:hover</code>	Aumenta a espessura do sublinhado dos links que o exibem ao passar o ponteiro. No menu, o feedback principal é a inversão de cores de <code>nav a:hover</code> .
<code>a:focus-visible,</code> <code>input:focus-visible,</code> <code>textarea:focus-visible</code>	Mostra um contorno afastado da caixa quando o navegador identifica a necessidade de foco visível, como na navegação por teclado. Não removemos o foco nativo sem oferecer um indicador substituto.
<code>address</code>	Remove o itálico padrão e separa os dados de contato do texto seguinte. O significado de contato do autor permanece o mesmo.
<code>form</code>	Separa o formulário do texto que explica sua natureza demonstrativa.
<code>fieldset</code>	Estiliza o agrupamento dos campos com borda, padding e cantos arredondados. <code>min-width: 0</code> permite que ele encolha sem impor a largura mínima intrínseca padrão do agrupamento.
<code>legend</code>	Cria espaço nas laterais do título do grupo e destaca sua identificação com cor e peso de fonte.
<code>label</code>	Torna o rótulo uma caixa <code>inline-block</code> para aceitar margem inferior e destaca o nome do campo em negrito. A associação <code>for/id</code> continua sendo feita pelo HTML.
<code>input, textarea</code>	Padroniza largura, padding, borda, fundo, texto e cantos dos controles. <code>font: inherit</code> faz os campos usarem a fonte do documento; <code>display: block</code> organiza cada campo em sua própria linha.

Seletor	Alvo e motivo das declarações
<code>textarea</code>	Define uma altura mínima para a mensagem e permite redimensionamento vertical, evitando que o visitante arraste o campo lateralmente para fora do formulário.
<code>button</code>	Harmoniza padding, borda, arredondamento e fonte com os demais controles.
<code>button:disabled</code>	Explicita o estado indisponível por cores e cursor. Quem impede a ativação é o atributo HTML <code>disabled</code> , não a regra CSS.
<code>footer</code>	Separa o encerramento com borda superior, texto centralizado e fonte menor. O rodapé continua no fluxo normal.
<code>footer > p:last-child</code>	Remove a margem inferior apenas do último parágrafo direto do rodapé; o padding do rodapé já garante o espaço final.

3.6. Cascata, especificidade e herança

O navegador combina as regras aplicáveis. `body` estabelece propriedades tipográficas que os descendentes normalmente herdam; `font: inherit` faz os controles de formulário participarem dessa escolha. Margens, bordas e padding precisam ser declarados nos elementos correspondentes porque não são herdados da mesma maneira.

`a` seleciona links em geral, enquanto `nav a` seleciona links dentro da navegação e tem maior especificidade. Por isso, os links do menu permanecem brancos mesmo com a regra geral de cor aparecendo depois no arquivo. Entre regras de mesma origem, importância e especificidade, a que aparece por último resolve conflitos sobre a mesma propriedade. Não há `!important` neste exemplo.

Seletores como `article > p` especializam o espaçamento definido em `p`; `header > p` especializa o parágrafo introdutório. `main li + li` deixa o primeiro item sem espaçamento adicional por selecionar apenas itens com um irmão anterior imediato. Assim, cada regra tem um alvo ligado à estrutura real da página.

3.7. Layout e limite desta etapa

O CSS usa o modelo de caixa: conteúdo, padding, borda e margem. `padding` afasta o texto da borda do próprio elemento; `margin` separa esse elemento dos vizinhos. `border-box` facilita o cálculo dos controles, pois a largura de 100% já inclui seu padding e suas bordas.

A imagem preserva a proporção com `height: auto`; a largura CSS substitui a largura de apresentação do atributo HTML quando a folha é aplicada. Os atributos originais continuam disponíveis na versão sem CSS. Os campos aproveitam a largura do formulário sem exigir tamanhos fixos em pixels.

Essas medidas evitam rigidez desnecessária, mas ainda não implementam o layout responsivo solicitado na etapa 3. Na próxima etapa, será preciso justificar a estratégia Mobile First, os breakpoints, o contêiner Grid ou Flexbox, a distribuição dos itens e a adaptação entre larguras.

3.8. Conferência e entrega

1. Abra `index.html` e confirme que o fundo, o cabeçalho, a fotografia circular e os blocos de conteúdo recebem os estilos.
2. Confira no navegador se `estilos.css` e a fotografia carregaram sem erro de caminho.
3. Clique nos seis destinos do menu e em “Voltar ao início”. A estilização não altera os IDs ou os destinos das âncoras.
4. Passe o ponteiro pelos links do menu e pelos links do conteúdo; observe os respectivos estados hover.
5. Use Tab para percorrer links e campos; verifique o contorno de foco. Clique nos rótulos e confira o foco no controle correspondente.
6. Digite nos campos, redimensione a mensagem verticalmente e confirme que o botão de envio permanece desabilitado.
7. Compare com a etapa 1: conteúdo e ordem de leitura devem coincidir; a transformação está na apresentação.
8. Antes de publicar, valide o HTML e o CSS em validadores de conformidade e registre a URL real do GitHub Pages na documentação da etapa.

4. Etapa 3: Mobile First e CSS Grid

4.1. Objetivo e requisitos

O objetivo desta versão é reconstruir o site segundo a estratégia Mobile First, configurar o viewport, evitar larguras rígidas, utilizar unidades adequadas, organizar o layout com Grid e acrescentar media queries para telas maiores. A imagem e os controles devem respeitar a largura disponível. A avaliação pede a conferência de navegação, texto e formulário em pelo menos três larguras.

O conteúdo pessoal e a ordem do HTML são os mesmos das etapas anteriores. Não há JavaScript, frameworks ou envio de formulário. Esta etapa concentra-se na adaptação entre larguras.

4.2. Arquivos e execução

```
03-mobile-first/  
|-- index.html  
|-- estilos.css  
|-- imagens/  
|   +-- bruno-honorato.jpg  
+-- README.md
```

Abra `index.html` no navegador. A folha de estilos e a fotografia são locais, e não há dependências para executar o exemplo. Cada etapa permanece independente para facilitar a comparação.

Aluno/autor do exemplo: Bruno de Castro Honorato Silva. Objetivo: adaptar a apresentação do site a diferentes larguras com Mobile First e CSS Grid. Link do site publicado: pendente; não foi realizada publicação no GitHub Pages nesta tarefa.

4.3. HTML e viewport

Nenhuma nova tag foi necessária para criar a grade. `main` já contém as seis `section` que serão seus itens; `nav` já contém a lista `ul`, cujos filhos `li` serão os itens da grade de navegação. O HTML foi preservado, com alteração apenas do título da aba para identificar a versão responsiva.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">  
<link rel="stylesheet" href="estilos.css">
```

`meta` configura o viewport: `width=device-width` aproxima sua largura em pixels CSS da largura do dispositivo, e `initial-scale=1.0` define a escala inicial. Sem essa configuração, navegadores móveis podem usar um viewport virtual amplo e reduzir a página, prejudicando a adaptação. Não bloqueamos o zoom.

`link` mantém a separação entre estrutura e apresentação: `rel="stylesheet"` identifica a folha e `href` indica seu caminho. `header`, `nav`, `main`, `section`, `article`, `figure`, `footer` e os elementos do formulário conservam os mesmos significados da etapa 1. CSS Grid altera a distribuição das caixas, sem mudar a semântica das tags.

Os IDs sobre, formacao, experiencias, habilidades, perfis e contato continuam sendo destinos do menu. Alguns também passam a ser selecionados no CSS para determinar a extensão na grade. Um atributo id pertence ao HTML; #contato em uma regra CSS é um seletor; href="#contato" é uma âncora. São usos relacionados, com funções diferentes.

4.4. Mobile First e breakpoints

As regras fora das media queries são a versão de celular: espaçamento externo de 1rem, títulos menores, menu em uma coluna e conteúdo em uma coluna. Essa é a experiência inicial, não uma versão desktop reduzida por exceções.

Largura de referência	Regras ativas com fonte inicial de 16 px	Menu	Conteúdo
375 px	Base	1 coluna	1 coluna
768 px	Base + min-width: 48rem	3 colunas	2 colunas
1200 px	Base + ambas as media queries	6 colunas	3 colunas

48rem corresponde aproximadamente a 768 px, e 75rem a 1200 px quando a fonte inicial do navegador é 16 px. Nas media queries, rem se refere ao tamanho inicial da fonte, não a uma alteração de font-size aplicada por CSS ao documento. Configurações do usuário podem mudar essa equivalência.

Escolhemos o primeiro breakpoint para permitir dois blocos de leitura e três links por linha; o segundo abre espaço para três blocos e seis links por linha. São decisões deste conteúdo, não regras universais sobre dispositivos. O limite global de 72rem evita que linhas de texto cresçam indefinidamente em monitores maiores.

Em 1200 px, as duas queries estão ativas. A de desktop aparece depois e substitui propriedades também definidas na de tablet, como a quantidade de colunas e o padding dos blocos centrais. As cores, bordas e regras que ela não altera continuam valendo.

4.5. Por que a grade funciona

```
main {
  display: grid;
  grid-template-columns: minmax(0, 1fr);
  gap: 1rem;
  align-items: start;
}

@media (min-width: 48rem) {
  main {
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 1.5rem;
  }
  #sobre, #contato {
    grid-column: 1 / -1;
  }
}

@media (min-width: 75rem) {
  main {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
  #experiencias, #contato {
    grid-column: span 2;
  }
}
```

`display: grid` transforma `main` em contêiner. Somente seus filhos diretos, as seis seções, viram itens dessa grade; os artigos dentro da seção de experiências continuam no fluxo normal.

`1fr` representa uma fração do espaço disponível, depois de considerar os intervalos. `repeat(2, ...)` e `repeat(3, ...)` criam colunas de mesma flexibilidade sem repetir manualmente a definição. `minmax(0, 1fr)` permite que a largura mínima de cada trilha seja zero, evitando que o mínimo intrínseco de um conteúdo longo force a grade a ultrapassar o contêiner. `min-width: 0` nas seções complementa essa decisão ao permitir que o próprio item encolha.

`gap` fornece espaço entre linhas e colunas. Por isso, removemos a antiga margem inferior das seções; manter as duas formas de separação somaria espaços desnecessários. `align-items: start` alinha os blocos no início de cada linha e evita esticar um bloco curto até a altura do vizinho. A altura de cada linha ainda é determinada pelo item mais alto, podendo haver espaço abaixo de uma seção mais curta.

`grid-column: 1 / -1` vai da primeira até a última linha explícita de colunas, cobrindo toda a largura. `grid-column: span 2` ocupa duas colunas a partir da posição definida pela colocação automática. No desktop, a regra de `#contato` substitui sua extensão de largura total definida no tablet. `#sobre` mantém a extensão total porque nenhuma regra posterior a substitui.

Distribuição esperada, seguindo a ordem do documento:

Celular (1 coluna):
Sobre
Formação
Experiências
Habilidades
Perfis
Contato

Tablet (2 colunas):
Sobre | Sobre
Formação | Experiências

Habilidades | Perfis
Contato | Contato

Desktop (3 colunas):

Sobre	Sobre	Sobre
Formação	Experiências	Experiências
Habilidades	Perfis	espaço livre
Contato	Contato	espaço livre

Repetir um nome no esquema indica a extensão de um único bloco, não conteúdo duplicado. Os espaços livres do desktop são intencionais: o contato precisa de duas colunas e não cabe na coluna restante da linha anterior. A colocação automática padrão avança para a próxima linha. Não usamos grid-auto-flow: dense para preencher esse espaço, pois isso poderia colocar visualmente conteúdos posteriores antes de outros e dificultar a correspondência com a leitura e a navegação por teclado.

A navegação é outra grade, independente de main. Seu contêiner é `nav ul`; os seis `li` passam de uma para três e depois seis colunas. Isso dá caixas de largura consistente aos links. `nav a` é um bloco que preenche a largura do item e tem altura mínima de `2.75rem`, aproximadamente `44 px` na configuração padrão, ampliando a área de interação. Não há menu escondido que dependa de JavaScript.

4.6. Seletores e finalidade de cada regra

A tabela cobre os seletores da folha de estilos. As regras repetidas nas media queries são detalhadas logo depois. Vírgulas agrupam alvos; espaço seleciona descendentes; `>` seleciona filhos diretos; `+` seleciona irmãos consecutivos; pseudoclasses selecionam estados ou posições.

Seletor	Por que é usado
<code>*</code>	Aplica border-box a todos os elementos, incluindo padding e bordas nas larguras declaradas.
<code>body</code>	Define fundo, cor, fonte e altura de linha; remove a margem padrão. <code>overflow-wrap: anywhere</code> permite quebrar conteúdo textual longo quando falta espaço.
<code>header, main, footer</code>	Usa <code>width: 100%</code> , largura máxima de <code>72rem</code> , margens automáticas e padding móvel de <code>1rem</code> para alinhar os blocos centrais sem largura rígida.
<code>header</code>	Preserva o fundo escuro, texto claro, alinhamento central e borda dourada da apresentação.
<code>h1, h2, h3</code>	Compartilha uma altura de linha compacta para títulos que precisem quebrar.
<code>h1</code>	Aplica tamanho móvel de <code>1.8rem</code> e margem inferior; cresce no tablet.
<code>header > p</code>	Estiliza somente a apresentação direta do cabeçalho, limitando seu comprimento de linha e aplicando texto claro.

Seletor	Por que é usado
<code>figure</code>	Remove recuos padrão e separa a fotografia do menu.
<code>figure img</code>	Aplica largura de 12rem, <code>max-width: 100%</code> e <code>height: auto</code> , preservando proporção e permitindo encolher. Centraliza a imagem como bloco e mantém retrato circular com borda dourada.
<code>figcaption</code>	Separa a legenda da foto e usa tamanho menor e texto claro.
<code>nav ul</code>	Cria a grade de links, inicialmente de uma coluna, com <code>gap: 0.5rem</code> , sem marcadores ou recuos.
<code>nav a</code>	Usa bloco, padding, altura mínima, borda e cantos arredondados para oferecer uma área clicável clara.
<code>nav a:hover</code>	Inverte fundo e texto quando o ponteiro passa sobre um destino do menu.
<code>main</code>	Cria a grade das seções, inicialmente de uma coluna, com gap e alinhamento no início das linhas.
<code>main > section</code>	Faz cada seção encolher com <code>min-width: 0</code> e delimita o bloco com padding, fundo branco, borda e arredondamento. A separação entre seções é responsabilidade do gap da grade.
<code>h2</code>	Marca cada seção com borda lateral, padding, margem inferior e tamanho móvel de 1.4rem, ampliado no tablet.
<code>h3</code>	Apresenta o título de cada projeto em tamanho subordinado ao título da seção.
<code>p</code>	Normaliza a margem dos parágrafos, preservando a distância inferior.
<code>main ul</code>	Mantém o recuo dos marcadores nas listas de conteúdo.
<code>main li + li</code>	Separa itens consecutivos sem acrescentar espaço antes do primeiro.
<code>article</code>	Delimita cada experiência com fundo claro, padding e borda lateral, sem transformá-la em uma nova grade.
<code>article + article</code>	Separa experiências consecutivas dentro da seção.
<code>article > p</code>	Remove a margem inferior do parágrafo direto; o padding do artigo já fornece espaço final.
<code>a</code>	Aplica cor aos links em geral e afasta o sublinhado do texto.
<code>a:hover</code>	Aumenta a espessura do sublinhado quando ele está visível.

Seletor	Por que é usado
<code>a:focus-visible,</code> <code>input:focus-visible,</code> <code>textarea:focus-visible</code>	Define contorno de foco afastado da caixa para navegação por teclado, sem depender de hover.
<code>address</code>	Mantém os dados de contato sem o itálico padrão e com espaço inferior.
<code>form</code>	Separa o formulário da explicação anterior.
<code>fieldset</code>	Agrupa campos com borda e padding móvel de 0.75rem; <code>min-width: 0</code> permite encolher abaixo do mínimo intrínseco.
<code>legend</code>	Usa <code>max-width: 100%</code> para respeitar a largura do grupo e destaca seu título com cor, peso e padding lateral.
<code>label</code>	Destaca os nomes dos campos e aceita margem inferior com <code>inline-block</code> . A associação com o controle continua sendo feita por <code>for</code> e <code>id</code> .
<code>input, textarea</code>	Define controles em bloco, com largura de 100%, padding, borda e fonte herdada. <code>border-box</code> impede que padding e borda aumentem essa largura para além do contêiner.
<code>textarea</code>	Mantém altura mínima para a mensagem e permite redimensionar apenas verticalmente.
<code>button</code>	Harmoniza borda, padding e fonte e limita a largura a 100%; <code>white-space: normal</code> permite quebrar o texto do botão em telas estreitas.
<code>button:disabled</code>	Comunica a indisponibilidade por cores e cursor. O atributo HTML <code>disabled</code> é responsável por impedir a ativação.
<code>footer</code>	Separa o encerramento com borda superior, alinhamento central e texto menor.
<code>footer > p:last-child</code>	Remove a margem final redundante do último parágrafo do rodapé.
<code>#sobre, #contato</code>	Na query de tablet, estende biografia e contato da primeira até a última linha de colunas.
<code>#experiencias, #contato</code>	Na query de desktop, reserva duas colunas para os relatos e para o formulário.

Na query `@media (min-width: 48rem)`, os seletores `header`, `main`, `footer` ampliam o padding para 1.5rem; `h1` passa a 2.4rem e `h2` a 1.6rem. `nav ul` passa a três colunas; `main` passa a duas e seu gap aumenta para 1.5rem. `main > section` amplia o padding para 1.5rem, e `fieldset` passa a 1rem. Os seletores de ID definem as extensões descritas acima.

Na query @media (min-width: 75rem), header, main, footer passam a padding de 2rem; nav ul passa a seis colunas e main a três. Os seletores de experiências e contato passam a duas colunas. Não é necessário repetir as regras de cor, fonte e borda, porque a cascata conserva as declarações anteriores.

nav li, existente na etapa 2, foi removido: os itens agora participam de Grid, que substitui inline-block e as margens dos itens como mecanismo de distribuição. Não adicionamos classes ou seletores que não tenham um alvo real no HTML.

4.7. Unidades, conteúdo e acessibilidade

rem acompanha a fonte raiz nas dimensões comuns; em acompanha a fonte do elemento no afastamento do sublinhado; % expressa a largura em relação ao bloco disponível; fr distribui o espaço da grade. Bordas de 1px permanecem finas, sem transformar o conteúdo em caixas de largura fixa.

A fotografia mantém seus atributos HTML width e height, mas o CSS controla a apresentação com largura flexível limitada e altura automática. O texto alternativo continua disponível. Não fixamos alturas de seções nem escondemos conteúdo que não caiba. As quebras de texto, minmax(0, 1fr) e min-width: 0 tratam causas de transbordamento em vez de ocultá-las com overflow-x: hidden.

Os links permanecem disponíveis em todas as larguras. Títulos, listas, rótulos e legendas preservam a leitura semântica. Não usamos order, posicionamento por áreas que reorganize as seções ou preenchimento denso da grade. A biografia precede formação, experiências, habilidades, perfis e contato na leitura do documento e na colocação visual.

O formulário continua uma demonstração: os campos recebem entrada, mas o botão desabilitado não envia nem confirma mensagens. O contato funcional oferecido é o link de e-mail, dependente de um aplicativo configurado.

4.8. Validação em três larguras

Abra as ferramentas de desenvolvimento do navegador, ative a simulação de dispositivos e use **375 px**, **768 px** e **1200 px** de largura. Com a fonte inicial padrão, confira a distribuição da tabela da seção 4.4. Teste também pouco antes e depois dos breakpoints, por exemplo 767/768 px e 1199/1200 px.

1. Confira nome, legenda e títulos sem cortes, fotografia sem ultrapassar o bloco e ausência de rolagem horizontal inesperada.
2. Clique nos seis links do menu e em “Voltar ao início”; todos devem apontar para a seção correta.
3. Percorra links e campos com Tab e verifique o foco visível e a ordem coerente.
4. Clique nos rótulos, digite nos campos e redimensione a mensagem verticalmente. O formulário deve respeitar a largura da seção em todas as três configurações.

5. Confira hover com o ponteiro no desktop e confirme que a navegação continua utilizável sem hover em uma interação por toque.
6. Aumente o zoom e verifique que o texto e o conteúdo continuam disponíveis; não há bloqueio de escala no viewport.
7. Salve uma captura de cada largura como evidência da entrega, se solicitada pelo professor.

Verificação realizada nesta implementação: inspeção estrutural do HTML, destinos de âncoras e rótulos, caminhos dos recursos locais, preservação do conteúdo e consistência das regras CSS. **A conferência visual nas três larguras e as capturas ainda precisam ser realizadas:** a validação registrada até aqui foi estrutural. As distribuições desta nota descrevem o comportamento definido pelo CSS, sem apresentar capturas ou resultados visuais que não foram obtidos.

Antes de publicar, passe os arquivos por validadores de conformidade HTML e CSS e registre o endereço real do GitHub Pages. Se publicar o repositório inteiro, o caminho desta etapa deverá terminar em `/07-trabalho-1/03-mobile-first/`, dependendo da configuração escolhida.

5. Código para consulta

O HTML da primeira etapa estabelece a estrutura comum. Nas etapas seguintes, acrescente a tag `link` no head, apontando para `estilos.css`, e ajuste o título da aba. As folhas CSS completas aparecem a seguir para relacionar cada seletor às explicações.

5.1. HTML da etapa 1

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Site pessoal de Bruno de Castro Honorato Silva,
professor da UTFPR: formação, pesquisa, tecnologias e contato.">
  <title>Bruno Honorato | Site pessoal</title>
</head>
<body>
  <header id="inicio">
    <h1>Bruno de Castro Honorato Silva</h1>
    <p>Professor da UTFPR, pesquisador e desenvolvedor de soluções para a Web.</p>
    <figure>
      
      <figcaption>Prof. Bruno Honorato - Campus Dois Vizinhos.</figcaption>
    </figure>
    <nav aria-label="Navegação principal">
      <ul>
        <li><a href="#sobre">Sobre mim</a></li>
        <li><a href="#formacao">Formação acadêmica</a></li>
        <li><a href="#experiencias">Experiências e projetos</a></li>
        <li><a href="#habilidades">Habilidades e tecnologias</a></li>
        <li><a href="#perfis">Perfis profissionais</a></li>
        <li><a href="#contato">Contato</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <section id="sobre">
      <h2>Sobre mim</h2>
      <p>Sou professor adjunto da Universidade Tecnológica Federal do Paraná (UTFPR).
Atuo com sistemas inteligentes para a Web, pagamentos digitais, geoinformática e
DevOps.</p>
      <p>Coordeno projetos de pesquisa, desenvolvimento e inovação em engenharia de
software, open banking, arquitetura Web em nuvem e dispositivos móveis. Busco
aproximar empresas e instituições de ensino e ampliar as oportunidades profissionais
dos estudantes.</p>
    </section>
    <section id="formacao">
      <h2>Formação acadêmica</h2>
      <ul>
        <li><strong>Doutorado em Sistemas e Computação</strong> - Universidade Federal
do Rio Grande do Norte (UFRN), 2017 a 2020.</li>
        <li><strong>Mestrado em Ciência da Computação</strong> - Universidade Estadual
do Ceará (UECE), 2011 a 2013.</li>
        <li><strong>Graduação em Análise e Desenvolvimento de Sistemas</strong> -
Faculdade Estácio do Ceará, 2008 a 2010.</li>
      </ul>
    </section>
    <section id="experiencias">
      <h2>Experiências e projetos</h2>
      <article>
        <h3>Ensino e pesquisa na UTFPR</h3>
        <p>Minha atuação reúne ensino de engenharia de software e pesquisa aplicada em
soluções Web, sistemas inteligentes e geoinformática.</p>
      </article>
      <article>
        <h3>Cooperação com o setor de pagamentos digitais</h3>
        <p>Contribuí para a implantação da FitBank no sertão cearense e liderei
projetos de pesquisa com investimentos do Goldman Sachs, conforme meu currículo
Lattes.</p>
      </article>
      <article>
        <h3>Otimização de rotas</h3>
        <p>No mestrado, pesquisei otimização de rotas com heurísticas em ambiente
georreferenciado. No doutorado, estudei uma variante do Problema do Caixeiro
Viajante com passageiros e tempo de coleta.</p>
      </article>
    </section>
  </main>
</body>
</html>
```

```

    </article>
</section>
<section id="habilidades">
  <h2>Habilidades e tecnologias</h2>
  <ul>
    <li>Desenvolvimento Web: HTML, CSS e JavaScript.</li>
    <li>Serviços e aplicações: Java, Spring e Python.</li>
    <li>Dados geográficos: PostgreSQL, PostGIS e GeoServer.</li>
    <li>Práticas de desenvolvimento: Git, Docker e DevOps.</li>
    <li>Pesquisa: otimização combinatória e engenharia de software.</li>
  </ul>
</section>
<section id="perfis">
  <h2>Perfis profissionais</h2>
  <ul>
    <li><a href="https://lattes.cnpq.br/6735984081484731">Currículo Lattes de Bruno Honorato</a></li>
    <li><a href="https://www.linkedin.com/in/bruno-honorato-3abb01180">LinkedIn de Bruno Honorato</a></li>
    <li><a href="https://orcid.org/0000-0003-1250-337X">ORCID de Bruno Honorato</a></li>
  </ul>
</section>
<section id="contato">
  <h2>Contato</h2>
  <address>
    Universidade Tecnológica Federal do Paraná - Campus Dois Vizinhos.<br>
    E-mail: <a href="mailto:brunosilva@utfpr.edu.br">brunosilva@utfpr.edu.br</a>
  </address>
  <p>Este formulário demonstra a estrutura HTML de coleta de dados. Não há serviço de envio; para entrar em contato, use o e-mail acima.</p>
  <form>
    <fieldset>
      <legend>Mensagem de contato (demonstração)</legend>
      <p><label for="nome">Nome:</label><br><input type="text" id="nome" name="nome" autocomplete="name" required></p>
      <p><label for="email">E-mail:</label><br><input type="email" id="email" name="email" autocomplete="email" required></p>
      <p><label for="mensagem">Mensagem:</label><br><textarea id="mensagem" name="mensagem" rows="5" cols="30" required></textarea></p>
      <button type="button" disabled>Envio indisponível nesta etapa</button>
    </fieldset>
  </form>
</section>
</main>
<footer>
  <p>Bruno Honorato - Site pessoal desenvolvido na disciplina Programação para Web 1.</p>
  <p><a href="#inicio">Voltar ao início</a></p>
</footer>
</body>
</html>

```

5.2. CSS da etapa 2

```
/* O modelo border-box inclui padding e bordas nas dimensões declaradas. */
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background-color: #eef2f6;
  color: #253247;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 1rem;
  line-height: 1.7;
  overflow-wrap: anywhere;
}

/* Os três blocos compartilham a mesma largura e o mesmo eixo central. */
header,
main,
footer {
  max-width: 58rem;
  margin: 0 auto;
  padding: 2rem;
}

header {
  background-color: #17324d;
  color: #ffffff;
  text-align: center;
  border-bottom: 0.35rem solid #e9b949;
}

h1,
h2,
h3 {
  line-height: 1.25;
}

h1 {
  margin: 0 0 1rem;
  font-size: 2.4rem;
}

header > p {
  margin: 0 auto 1.5rem;
  max-width: 40rem;
  color: #dce7f2;
}

figure {
  margin: 0 0 1.5rem;
}

figure img {
  display: block;
  width: 12rem;
  max-width: 100%;
  height: auto;
  margin: 0 auto;
  border: 0.25rem solid #e9b949;
  border-radius: 50%;
}

figcaption {
  margin-top: 0.75rem;
  font-size: 0.9rem;
  color: #dce7f2;
}

nav ul {
```

```

    margin: 0;
    padding: 0;
    list-style: none;
}

nav li {
    display: inline-block;
    margin: 0.25rem;
}

nav a {
    display: inline-block;
    padding: 0.55rem 0.8rem;
    color: #ffffff;
    border: 1px solid #8ea8c1;
    border-radius: 0.35rem;
    text-decoration: none;
}

nav a:hover {
    background-color: #ffffff;
    color: #17324d;
}

main > section {
    margin-bottom: 1.5rem;
    padding: 1.5rem;
    background-color: #ffffff;
    border: 1px solid #ccd6e0;
    border-radius: 0.5rem;
}

h2 {
    margin: 0 0 1rem;
    padding-left: 0.75rem;
    border-left: 0.25rem solid #a66c00;
    color: #17324d;
    font-size: 1.6rem;
}

h3 {
    margin: 0 0 0.75rem;
    color: #17324d;
    font-size: 1.15rem;
}

p {
    margin: 0 0 1rem;
}

main ul {
    margin: 0;
    padding-left: 1.5rem;
}

main li + li {
    margin-top: 0.75rem;
}

article {
    padding: 1rem;
    background-color: #f4f7fa;
    border-left: 0.2rem solid #8ea8c1;
}

article + article {
    margin-top: 1rem;
}

article > p {
    margin-bottom: 0;
}

```

```

a {
  color: #155a8a;
  text-decoration-offset: 0.2em;
}

a:hover {
  text-decoration-thickness: 0.15em;
}

/* O foco precisa continuar visível também para quem usa o teclado. */
a:focus-visible,
input:focus-visible,
textarea:focus-visible {
  outline: 0.2rem solid #a66c00;
  outline-offset: 0.2rem;
}

address {
  margin-bottom: 1rem;
  font-style: normal;
}

form {
  margin-top: 1.5rem;
}

fieldset {
  min-width: 0;
  margin: 0;
  padding: 1.25rem;
  border: 1px solid #8ea8c1;
  border-radius: 0.35rem;
}

legend {
  padding: 0 0.5rem;
  color: #17324d;
  font-weight: bold;
}

label {
  display: inline-block;
  margin-bottom: 0.35rem;
  font-weight: bold;
}

input,
textarea {
  display: block;
  width: 100%;
  padding: 0.75rem;
  border: 1px solid #64748b;
  border-radius: 0.25rem;
  background-color: #ffffff;
  color: #253247;
  font: inherit;
}

textarea {
  min-height: 8rem;
  resize: vertical;
}

button {
  padding: 0.75rem 1rem;
  border: 1px solid #8ea8c1;
  border-radius: 0.25rem;
  font: inherit;
}

button:disabled {
  background-color: #e2e8f0;
}

```

```
    color: #475569;
    cursor: not-allowed;
}

footer {
  border-top: 1px solid #ccd6e0;
  text-align: center;
  font-size: 0.9rem;
}

footer > p:last-child {
  margin-bottom: 0;
}
```

5.3. CSS da etapa 3

```
/* O modelo border-box inclui padding e bordas nas dimensões declaradas. */
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background-color: #eef2f6;
  color: #253247;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 1rem;
  line-height: 1.7;
  overflow-wrap: anywhere;
}

/* Mobile First: uma coluna e espaços menores são a experiência base. */
header,
main,
footer {
  width: 100%;
  max-width: 72rem;
  margin: 0 auto;
  padding: 1rem;
}

header {
  background-color: #17324d;
  color: #ffffff;
  text-align: center;
  border-bottom: 0.35rem solid #e9b949;
}

h1,
h2,
h3 {
  line-height: 1.25;
}

h1 {
  margin: 0 0 1rem;
  font-size: 1.8rem;
}

header > p {
  margin: 0 auto 1.5rem;
  max-width: 40rem;
  color: #dce7f2;
}

figure {
  margin: 0 0 1.5rem;
}

figure img {
  display: block;
  width: 12rem;
  max-width: 100%;
  height: auto;
  margin: 0 auto;
  border: 0.25rem solid #e9b949;
  border-radius: 50%;
}

figcaption {
  margin-top: 0.75rem;
  font-size: 0.9rem;
  color: #dce7f2;
}
```

```

nav ul {
  display: grid;
  grid-template-columns: minmax(0, 1fr);
  gap: 0.5rem;
  margin: 0;
  padding: 0;
  list-style: none;
}

nav a {
  display: block;
  min-height: 2.75rem;
  padding: 0.55rem 0.8rem;
  color: #ffffff;
  border: 1px solid #8ea8c1;
  border-radius: 0.35rem;
  text-decoration: none;
}

nav a:hover {
  background-color: #ffffff;
  color: #17324d;
}

main {
  display: grid;
  grid-template-columns: minmax(0, 1fr);
  gap: 1rem;
  align-items: start;
}

main > section {
  min-width: 0;
  padding: 1rem;
  background-color: #ffffff;
  border: 1px solid #ccd6e0;
  border-radius: 0.5rem;
}

h2 {
  margin: 0 0 1rem;
  padding-left: 0.75rem;
  border-left: 0.25rem solid #a66c00;
  color: #17324d;
  font-size: 1.4rem;
}

h3 {
  margin: 0 0 0.75rem;
  color: #17324d;
  font-size: 1.15rem;
}

p {
  margin: 0 0 1rem;
}

main ul {
  margin: 0;
  padding-left: 1.5rem;
}

main li + li {
  margin-top: 0.75rem;
}

article {
  padding: 1rem;
  background-color: #f4f7fa;
  border-left: 0.2rem solid #8ea8c1;
}

```

```

article + article {
  margin-top: 1rem;
}

article > p {
  margin-bottom: 0;
}

a {
  color: #155a8a;
  text-decoration-offset: 0.2em;
}

a:hover {
  text-decoration-thickness: 0.15em;
}

/* O foco precisa continuar visível também para quem usa o teclado. */
a:focus-visible,
input:focus-visible,
textarea:focus-visible {
  outline: 0.2rem solid #a66c00;
  outline-offset: 0.2rem;
}

address {
  margin-bottom: 1rem;
  font-style: normal;
}

form {
  margin-top: 1.5rem;
}

fieldset {
  min-width: 0;
  margin: 0;
  padding: 0.75rem;
  border: 1px solid #8ea8c1;
  border-radius: 0.35rem;
}

legend {
  max-width: 100%;
  padding: 0 0.5rem;
  color: #17324d;
  font-weight: bold;
}

label {
  display: inline-block;
  margin-bottom: 0.35rem;
  font-weight: bold;
}

input,
textarea {
  display: block;
  width: 100%;
  padding: 0.75rem;
  border: 1px solid #64748b;
  border-radius: 0.25rem;
  background-color: #ffffff;
  color: #253247;
  font: inherit;
}

textarea {
  min-height: 8rem;
  resize: vertical;
}

```

```

button {
  max-width: 100%;
  white-space: normal;
  padding: 0.75rem 1rem;
  border: 1px solid #8ea8c1;
  border-radius: 0.25rem;
  font: inherit;
}

button:disabled {
  background-color: #e2e8f0;
  color: #475569;
  cursor: not-allowed;
}

footer {
  border-top: 1px solid #ccd6e0;
  text-align: center;
  font-size: 0.9rem;
}

footer > p:last-child {
  margin-bottom: 0;
}

/* Tablet: duas colunas de leitura, com biografia e contato em largura total. */
@media (min-width: 48rem) {
  header,
  main,
  footer {
    padding: 1.5rem;
  }

  h1 {
    font-size: 2.4rem;
  }

  h2 {
    font-size: 1.6rem;
  }

  nav ul {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }

  main {
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 1.5rem;
  }

  main > section {
    padding: 1.5rem;
  }

  #sobre,
  #contato {
    grid-column: 1 / -1;
  }

  fieldset {
    padding: 1rem;
  }
}

/* Desktop: três colunas; relatos e formulário recebem duas delas. */
@media (min-width: 75rem) {
  header,
  main,
  footer {
    padding: 2rem;
  }
}

```

```
nav ul {  
  grid-template-columns: repeat(6, minmax(0, 1fr));  
}  
  
main {  
  grid-template-columns: repeat(3, minmax(0, 1fr));  
}  
  
#experiencias,  
#contato {  
  grid-column: span 2;  
}  
}
```